SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Specification-Driven Binary Translator

Robert Barinov

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Specification-Driven Binary Translator

Spezifikationsgesteuerter Binärübersetzer

Author: Robert Barinov
Examiner: Dr. Pramod Bhatotia
Supervisor: Wonkeun Choi, Martin Fink
Submission Date: 04.03.2026

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 04.03.2026 Robert Barinov

Acknowledgments

I want to thank my supervisors, Anders and Martin, for patiently guiding me throughout the project and supporting me along the way. I am glad to have had the opportunity to work with you and to learn from you; it's invaluable to me.

Also, thanks to my colleagues at Intel for sparking my interest in compilers.

And to my family — thank you for always being there.

Abstract

Static binary translation is an important technique for enabling cross-architectural compatibility. At its core lies the challenge of building a binary lifter — a task that, when done manually, is both cumbersome and error-prone. The release of ARM's Machine Readable Architecture Specification (MRAS), a formal, machine-readable description of ARM architecture semantics, opened the door to a new approach: generating binary lifters directly from the architecture specification.

In this thesis, we present SGTranslator, a specification-driven static binary translator from AArch64 to x86-64. SGTranslator is built on top of Airlift — a specification-driven lifter from AArch64 to LLVM IR. We extend it to support a broader class of programs, adding handling for position-independent code, shared library calls, and system calls. We complement this with a helper runtime library that enables the execution of the resulting binaries on the host machine.

We evaluate SGTranslator by translating a benchmark suite, executing the translated binaries, and comparing their performance against native x86-64 executables and QEMU. On average, translated binaries run 3.7 times slower than their native counterparts and are slower than QEMU across our benchmarks.

These results are not unexpected, since Airlift was designed as a proof-of-concept for specification-driven binary translation rather than a performance-optimized tool. As such, the performance gap reflects its current limitations rather than a ceiling on the approach. A more performance-conscious specification-driven lifter would open the path to translations that are both correct and efficient.

Contents

Acknowledgments **iii**

Abstract **iv**

1 Introduction **1**

2 Background **3**
2.1 Binary Translation 3
2.1.1 Dynamic Binary Translation 4
2.1.2 Static Binary Translation 4
2.2 Binary Lifting 4
2.2.1 LLVM 5
2.2.2 AArch64 5
2.3 ELF File Format 5
2.3.1 Dynamic Linking 7

3 Overview **9**

4 Design **11**
4.1 Lifting Instructions 11
4.2 Guest Executable Organization 13
4.3 Functions 13
4.3.1 Function Declarations 14
4.3.2 Internal Function Calls 14
4.3.3 External Function Calls 17
4.4 System Calls 19
4.5 Position-Independent Code 20
4.6 Specifics of C translation 22
4.7 Limitations 23

5 Implementation **24**

6 Evaluation **26**
6.1 Performance 26
6.1.1 Methodology 26
6.1.2 Results 28
6.2 Correctness 30

7 Related Work **33**

8 Conclusion **35**
8.1 Future Work 35

Abbreviations **36**

List of Figures **37**

List of Tables **38**

Bibliography **39**

1 Introduction

As the IT industry expands and evolves, it introduces new processor architectures and execution models. Contemporary computing platforms include a wide range of instruction set architectures (ISAs), such as x86-64, ARM, MIPS, and PowerPC. Each of these architectures addresses particular design goals — performance, power efficiency — and occupies a distinct role within the computing ecosystem. A natural consequence of this architectural diversity is the problem of compatibility. Software distributed only in binary form is tightly coupled to the instruction set architecture (ISA) for which it was compiled. When the corresponding source code is unavailable, recompilation for a different architecture is impossible, and alternative approaches must be considered.

Binary translation directly addresses this problem. It involves analyzing the executable to be translated, lifting it to an intermediate representation (IR), and recompiling it to the target architecture. To lift an executable, the problem of writing a binary lifter must be solved. The goal of a binary lifter is to take a binary executable as an input and produce a compiler IR module that will mimic the behavior of the input executable.

Writing binary lifters manually is bug-prone and cumbersome. To write a function that will lift an instruction, one must write a decoder that decodes the instructions of a binary and a code generator that generates IR equivalent to the decoded instruction's semantics, described in the architecture specification. If the target ISA is ever expanded by its authors (e.g., RISC-V and ARM extensions), the lifter must be manually extended to support the changes to the ISA.

A possible alternative approach is to generate the lifter directly from the ISA's specification. This is exactly the problem addressed in a previous Master's thesis. Since 2017, ARM's ISA specification has been available in machine-readable form as the ARM Machine Readable Architecture Specification (MRAS) [Rei17]. The previous thesis involved building Airlift — a specification-driven, MRAS-based binary lifter from AArch64 to LLVM IR [Cho25].

A complete static binary translator has components beyond a lifter. An executable cannot be lifted back to an architecture-independent representation, at least because of the system calls it invokes, the set of which does not necessarily match across different architectures. The solution is lifting to an IR with architecture-specific placeholders and introducing a compatibility layer that handles them at runtime — the runtime helper library. It is also necessary to lower the resulting IR to the target architecture and to

provide a way for it to interact with the runtime library.

These are exactly the problems this thesis solves. We develop a functional binary translator from AArch64 to x86-64, with the following core contributions:

- Modifying the lifter to support translation of a larger subset of programs. The modifications to the lifter enable the translator to handle features such as position-independent code and dynamic function calls.
- Developing a runtime library, a necessary compatibility interface for the guest binary. The library handles guest executable state initialization and system call emulation.
- Integrating all components into a single executable, SGTranslator. SGTranslator is also responsible for patching the lifted IR with interfaces that enable its communication with the runtime helper library.

The thesis demonstrates that a binary translator, core parts of which have been generated from an ISA specification, can be a plausible solution to the problem of reliable binary translation. To demonstrate the features SGTranslator is able to handle, and to demonstrate its correctness, we develop a manually written test suite and test our translator on it. To evaluate SGTranslator, we run it on a benchmark suite and assess the performance of the resulting binaries.

2 Background

To successfully develop a binary translator, we must first understand both the system we are building and the underlying theory behind the concepts we use, such as the structure and layout of an ELF file.

2.1 Binary Translation

As processors have evolved, numerous instruction set architectures (ISAs) have emerged from the industry. Until recently, the complex instruction set computer (CISC) architecture has been the dominant paradigm in desktop computing, most notably, Intel's x86-64. However, in recent years, the reduced instruction set computer (RISC) architecture, particularly the ARM architecture, has gained greater popularity in the desktop segment. After Apple introduced its M-series chips, ARM saw a surge in popularity in the desktop segment [App20]. Apple's new processors became a major competitor to x86-64, which Apple had used in its MacBooks before its deal with ARM. RISC processors have gained popularity thanks to their power effectiveness and reduced complexity compared to CISC architectures.

In this increasingly diverse market, software compatibility has become more important. Programs written and compiled for x86-64 do not run natively on ARM, and vice versa. Recompiling the source code for another architecture may not always be an option, since the source code may have been lost or written in a legacy language whose compiler does not support the (newer) target architecture. Even if the source code is available and the toolchain supports the target architecture, the source code might be target-specific (e.g, containing Intel's SSE intrinsics in the case of x86-64 [Int24]), which makes recompilation of it to another target non-trivial [CM96]. Despite these problems, compatibility is still desired.

An alternative approach to recompilation is binary translation — a technique that involves translating the instructions of a binary executable compiled for one ISA into semantically equivalent instructions of another ISA, without referring to the source code. There are two primary approaches: static binary translation (SBT) and dynamic binary translation (DBT).

2.1.1 Dynamic Binary Translation

Dynamic binary translation refers to the process of translating a program compiled for one ISA into instructions for another ISA and executing them on the go. The following is a rough outline of the operation of a dynamic binary translator (DBT).

First, the DBT initializes the architectural state of the guest application (the application we are dynamically translating). The state consists of the guest application's stack and registers. After that, the DBT finds the entry point of the guest application and starts translating the guest architecture's instructions to sequences of native ISA instructions, usually until the next jump. This means that the DBT is translating at the basic block granularity. A basic block is an instruction sequence with no jumps, except for the jump that leads out of it to the next basic block. The DBT caches program sections that have already been translated and reuses them on demand.

Instead of jump instructions, the DBT emits calls to a dispatcher, a function responsible for redirecting the binary translator's control flow to instructions located at the jump address. From there, the DBT continues translating the program. Another responsibility of the DBT is to emulate the system calls made by the guest program. Whenever the guest application makes a system call, the DBT intercepts it and translates it to a corresponding system call on the native architecture [Pro02].

A drawback of DBT is the overhead incurred by simultaneous translation and execution.

2.1.2 Static Binary Translation

An alternative to dynamic binary translation is static binary translation. Static binary translation is the process of analyzing an executable for one architecture and producing an executable for another that behaves equivalently, ahead of time. A static binary translator (SBT) will disassemble the source executable, translate (lift) it to an IR, and lower it to the target architecture [CM96]. Static binary translation is limited, as it cannot generally handle executable features such as code/data interleaving, self-modifying code, and indirect jumps. However, these limitations do not make SBT completely impractical. One could statically translate as much of the binary as possible and fall back to dynamic translation for parts of the code for which accurate ahead-of-time translation is not possible. This approach is referred to as hybrid binary translation [She+12].

2.2 Binary Lifting

Writing a static binary translator involves solving the problem of writing a binary lifter. A binary lifter is a program that takes a binary executable as an input and lifts it to a

compiler IR. The idea is that the resulting IR can be analyzed using existing compiler tooling and, with some caveats that we will discuss later, lowered to the desired target architecture. Our project builds upon an already existing binary lifter, Airlift [Cho25].

2.2.1 LLVM

An intermediate representation (IR) is a language used internally by a compiler to represent a program, agnostically to the language from which it was generated [Liu+22]. A well-designed IR is suitable for analysis, subsequent optimization, and lowering to a target architecture. There are different IRs available, such as GNU GCC's GIMPLE or LLVM IR [GNU] [The26b].

For our project, we used the LLVM Project's LLVM IR, since Airlift is built on it. The LLVM project is an open-source compiler framework; one of its most prominent features is the IR it offers. The LLVM Project offers many analysis and optimization tools, a programmatic interface for its IR, and allows compilation to multiple backends. These tools were utilized extensively to improve the performance of the binaries we generate [LA04].

2.2.2 AArch64

AArch64 is a 64-bit RISC architecture developed by ARM. The AArch64 instructions are 32-bit wide. In 2017, ARM released a machine-readable specification for their architectures, the MRAS. All future architectures thereafter used Architecture Specification Language (ASL) to describe their semantics [Rei17]. Machine-readability enables automatic generation of tools that work with ARM, such as simulators and binary translators, and facilitates their formal verification. Airlift, the lifter from AArch64 to LLVM IR, that we use in our project, is generated from the MRAS [Cho25].

2.3 ELF File Format

The executable and linkable format (ELF) is a common executable file format on Linux. An ELF object file can be of three main types: an executable file, a relocatable (linkable) file, and a shared library. To navigate an ELF file, one must start by reading its header. The header is at the start of the file and is the only position-wise fixed component of an ELF file.

The ELF header contains general information about an object file, such as its type (executable/relocatable/shared), the underlying architecture, and other metadata. It also contains offsets (relative to the ELF's start) of the two most important data structures in an ELF file — the section header table and the program header table.

The section header table contains information about an object's sections — the indivisible units of an ELF file. A section header is a structure containing the information required to work with the section it describes, such as the section's starting address and the section's size. Any ELF file has sections and a section header table.

The second key structure, present only in executables and shared libraries, is the program header table. Its purpose is to define how the sections of a program are mapped into RAM — that is, to describe the memory layout of the program at runtime. The table consists of program headers, each of which describes a segment, a contiguous portion of the program's memory image.

Each program header contains information such as the segment type, its file offset, file size, virtual address, virtual size, and access flags. The flags specify memory permissions, including readability, writability, and executability, which the segment receives from the loader. It is important to distinguish between a segment's file offset and its runtime virtual address: the file offset indicates the location in the object file from which the segment data should be copied, while the virtual address specifies where the segment will reside in memory.

The most important segment type is `PT_LOAD`. Segments of this type are loadable segments, which the loader maps into memory before the program starts. The result of mapping all segments into memory is called the process memory image. When the loader maps a segment into memory, it copies data from the ELF file in the range `[file_start ... file_start + file_size]` into the virtual memory range `[virtual_start ... virtual_start + virtual_size]`. The sizes of these ranges must not match; for instance, the `.bss` section occupies no space in an ELF file on disk, but must be mapped to a memory chunk of size specified by the program header [Ora10].

Figure 2.1 shows a simplified version of the segment loading process (alignment and other fields are left out). A file segment **S** consisting of sections S1 and S2 is being loaded into virtual memory (mapped to the virtual segment **V**). On the right, a program header is shown. **S** is a loadable segment, since `p_type` is `PT_LOAD`. **S** is readable and writable, which is shown in `p_flags`. In the file, **S** starts at offset `p_offset` and spans `p_filesz` bytes. The file segment **S** is mapped into virtual memory to its corresponding virtual segment **V**, starting at virtual address `p_vaddr` and spanning `p_memsz` bytes. Since **S** is smaller in size than **V**, some parts of the virtual segment are zeroed out, which is shown by the red-filled section of the process image. The file-backed, non-zero part of the virtual segment **V** ends at `p_vaddr` + `p_filesz`.

An ELF file contains special sections called symbol tables. Symbol tables hold strings that are names of different entities in the object file. One of them is `.shstrtab` - the section header string table. The section header string table holds strings that are names of sections. Its offset in the section header table can be obtained from a dedicated ELF

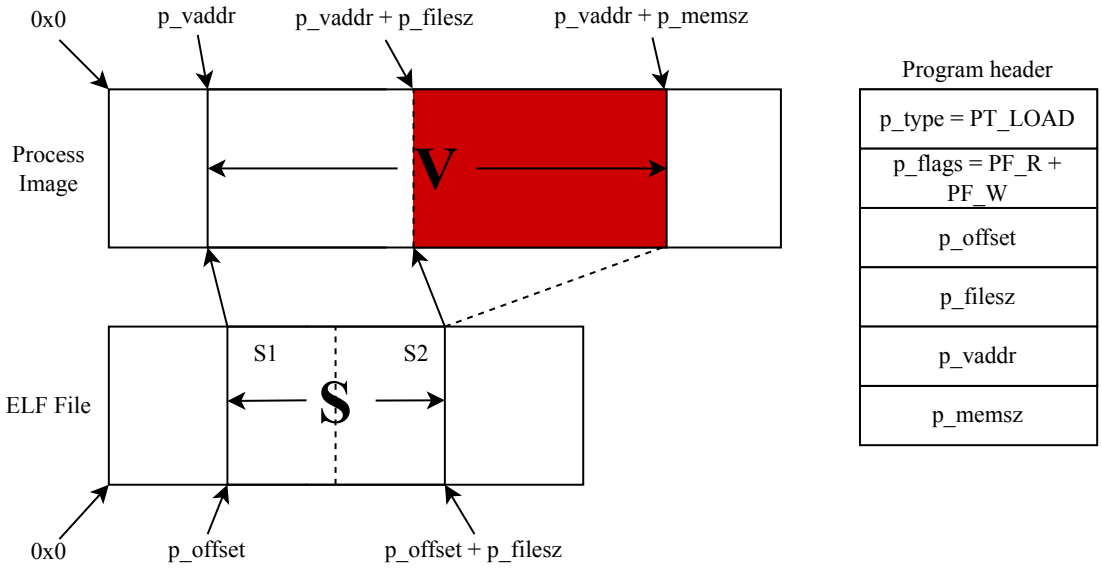

Figure 2.1: Loading of a segment according to a program header. S1 and S2 are sections, **S** is the segment containing them. The filled red part of **V** is the part of the virtual segment that is zeroed out, since it is not backed by data from the file segment.

header field. The symbol table can be used to extract function names from a binary, which will be useful to us.

2.3.1 Dynamic Linking

Linking is the process of combining multiple relocatable object files into a single executable. It is performed by a linker. There are two types of linking: static and dynamic. When an executable is linked statically, its dependencies are embedded into it.

On the other hand, there are dynamically linked binaries. When a binary is linked dynamically, rather than containing its dependencies directly, it contains references to them. A dependency of a dynamically linked executable is called a shared library. The ELF format supports dynamic linking: an ELF file can depend on shared libraries, be a shared library, or both at the same time. When a dynamically linked program is loaded into memory before execution, all shared libraries that it references are also loaded along with it.

The shared libraries may be loaded to different, unpredictable addresses across executions. This means that the references to them have to be handled specially. These

references are handled through relocations — records produced by the linker that tell the dynamic loader which addresses need to be filled in, and how, once the final load addresses are known. For this, two special structures are created at link-time, the global offset table (GOT) and the procedure linkage table (PLT). The GOT is a structure that all dynamic data symbols refer to after linking. At load time, it is filled with the absolute addresses of the dynamic data symbols. This is not the case for shared library functions. They are resolved lazily through PLT, which means a GOT entry is filled in only after a function has been called for the first time. For brevity, we will omit the details on how the PLT mechanism works. It is important for us to understand that encountering a call stubbed with a PLT entry in a binary means that it is a call to a shared library function [H J+21].

There are many benefits to dynamic linking. When programs are linked dynamically, the same dependency does not need to be redundantly stored on disk (as part of the executables that depend on it), unlike with static linking. It suffices to have a single copy of the shared library, which then can be loaded on demand. In practice, it also helps save RAM, since multiple processes that depend on a shared library can refer to the same physical pages when accessing it, at least to the ones that are read-only [IBM16].

3 Overview

In this thesis, we build SGTranslator — a static specification-driven binary translator from AArch64 to x86-64. SGTranslator comprises three core components. In this section, we briefly introduce them and explain how they work together as a static binary translator.

The first and most substantial of them is our lifter **Airlift**, which lifts an executable's AArch64 instructions to LLVM IR. Airlift is a specification-driven lifter, meaning its core components are generated from an architecture specification and not written manually. To be more specific, they are generated from ARM's MRAS. The lifter generation procedure involves taking the ASL description of each machine instruction and generating Rust code that emits a correct LLVM IR sequence upon encountering that instruction. Airlift is modified in multiple ways to work correctly as a component of SGTranslator. We discuss the modifications and their reasoning in detail in chapter 4.

The second component we develop is the **translator wrapper** itself. As we discussed before, a binary lifter, in our case Airlift, is not a binary translator on its own. The lifter only lifts our executable to an IR. To obtain the target-architecture executable, the IR must be compiled down to the target architecture. This is one of the translator wrapper's responsibilities, along with optimizing the IR and transforming it as needed to ensure correct execution.

The final component of our project is the helper runtime library, the **binary translator runtime (BTR)**. The BTR is an interface between the guest executable and the host architecture. Its function is to translate the guest binary's system calls into the host system's system calls and to initialize the guest executable's state, such as registers and the stack, before it starts executing. This is akin to how a DBT initializes the guest executable's state before translating and running it.

Figure 3.1 depicts the general workflow of the translator wrapper. Every component we develop in this project is marked with a separate color in the figure.

As an input, the translator wrapper takes the native AArch64 binary, the path to the BTR, C runtime dependencies, such as `crt1` and other initialization modules, paths to dynamic libraries the guest binary depends on, and paths to their headers. The latter two are required for translating binaries that depend on dynamic libraries. We discuss this topic in detail in section 4.3.3. The guest binary is lifted by the lifter into

LLVM IR, after which the patcher patches the LLVM IR module with a new entry point and removes the stubs that were left by the lifter to translate external calls (4.3.3). After being patched, the executable is optimized by LLVM's optimizer and lowered to x86-64 by LLVM's x86-64 backend. The linker links the lowered object file with the C runtime, the BTR, and its dynamic library dependencies. The result of this process is the translated binary, which can be run natively on x86-64.

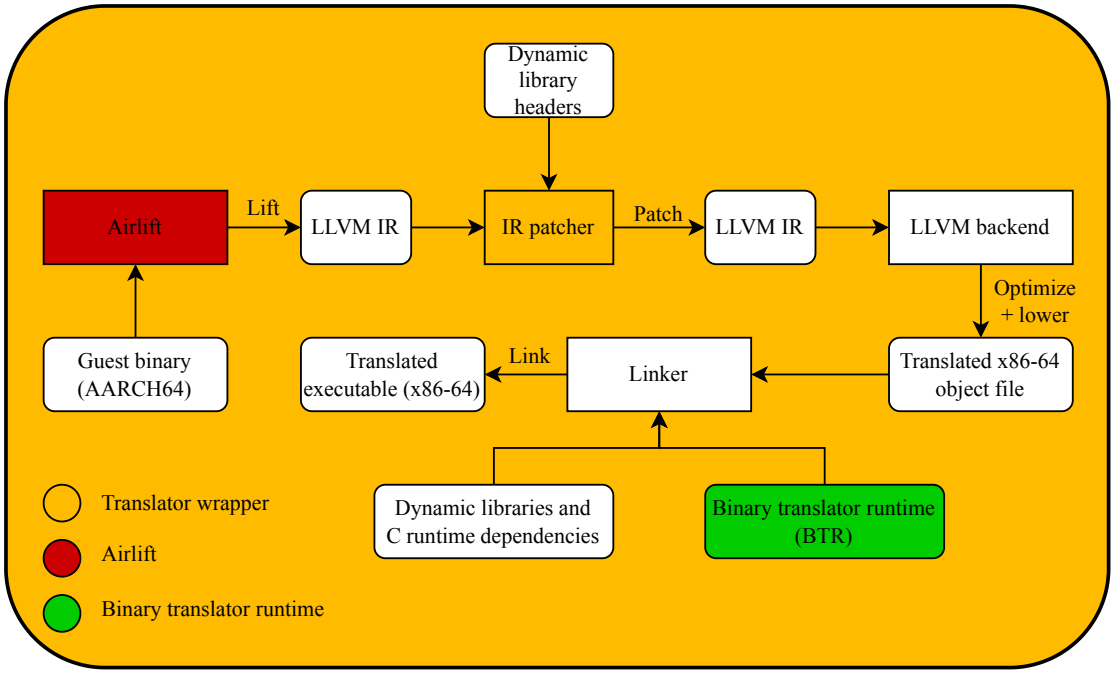

Figure 3.1: Overview of the translator workflow

4 Design

This chapter will cover the modifications we made to Airlift, as well as the techniques that we used to expand the set of programs we can translate. It will also explain the purpose of the runtime helper library and the changes made to IR by the patcher to make it work with the runtime helper library.

4.1 Lifting Instructions

We will start with a brief explanation of how Airlift lifts instructions. When a processor executes an instruction, it not only operates on its direct operands but also induces side effects, such as the modification of condition flags. This means that the processor has a state, and that executing the same instruction in two different states may lead to different behaviors.

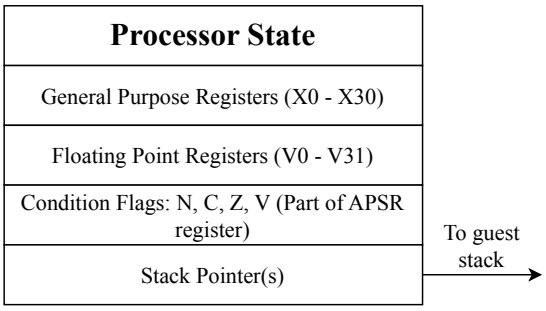

Figure 4.1: A simplified version of a processor state

To emulate the processor state, we introduce the concept of a guest binary state (or simply guest state). Figure 4.1 shows a (simplified) possible representation of the guest binary state. It consists of general registers (`X0`–`X30` on AArch64), floating-point registers (`V0`–`V31`), condition flags, and stack pointers, one for each privilege level. In practice, we use only one stack pointer that points to the guest stack (explained in later sections), since we are not emulating the full system. Condition flags are required to implement features such as conditional branching in hardware. They are set by instructions such as `subs` — the flag-updating subtraction instruction in AArch64.

When a `subs` instruction is executed by the processor, it checks whether the result is negative, zero, resulted in a carry condition or a signed overflow condition and sets the condition flags accordingly. A subsequent conditional branch instruction, e.g. `b.eq` — the branch if equal instruction in AArch64, will then read these flags to decide whether the instruction pointer should be changed or not. The `b.eq` will branch if APSR — the register that packs flags together on AArch64, has the bit corresponding to the zero flag set. In the lifter, we model the guest state as a thread-local structure. The lifter refers to it whenever an instruction it is lifting accesses the architectural state.

For instance, when the lifter encounters an instruction with register operands, it is lifted to a sequence of LLVM IR instructions that starts by loading from the fields of the guest state corresponding to the registers of the original instruction. If the instruction has a register destination, the sequence stores into the corresponding guest state field. Effectively, when we run the translated binary on x86-64, we emulate the source binary.

An example of this approach is shown in figure 4.2, which shows a translation scheme for the AArch64 `add` instruction.

```
add x0, x1, #42
```

(a) AArch64 add instruction

```
%ptr_x1 = getelementptr { { i1, i1, i1, i1 }, [31 x i64], i64, i64, i64, i64, [32 x
    ↪ i2048], [16 x i256], i256 }, ptr %state, i32 0, i32 1, i32 1 ; Get the pointer
    ↪ to x1 in state
%ld_x1 = load i64, ptr %ptr_x1, align 8 ; Dereference pointer to x1
%result = add i64 %ld_x1, 42 ; Add 42 to the dereferenced value
%ptr_x0 = getelementptr { { i1, i1, i1, i1 }, [31 x i64], i64, i64, i64, i64, [32 x
    ↪ i2048], [16 x i256], i256 }, ptr %state, i32 0, i32 1, i32 0 ; Get the pointer
    ↪ to x0 in state
store i64 %result, ptr %ptr_x0 ; Store the result of the addition to x0
```

(b) Lifted code

Figure 4.2: Lifted AArch64 add instruction

Figure 4.2a shows an AArch64 `add` instruction, which adds the value stored in register `x1` with 42 and stores the result into `x0`. Below it, figure 4.2b shows an LLVM IR sequence similar to one our lifter would emit when lifting this instruction, with details left out for brevity. We start by retrieving the address of the field corresponding to `x1` from our state structure. After that, we load from the address we computed and add 42 to the loaded value. We fetch the field corresponding to `x0` and store into it. After this sequence, the guest state is updated the same way the architectural state

would be updated if 4.2a was executed natively.

The style of IR emitted by Airlift is known as emulation-style IR, as opposed to high-level IR. In emulation-style IR, every machine instruction is lifted into a sequence of statements that try to emulate the machine execution [Liu+22]. We chose the emulation-style IR approach because the project's goal is to demonstrate the feasibility of a specification-driven binary translator. A drawback of this approach is that we have a memory access overhead for every instruction that reads or writes to the guest state.

4.2 Guest Executable Organization

Before the guest binary can start executing, some initialization must be performed. The binary translator has to allocate a stack for the guest binary to use. Due to differences in calling conventions, it is more convenient for the guest binary to use its own stack rather than reuse the host machine's native stack.

Whenever the lifter lifts a guest binary's function, which we will discuss in detail in section 4.3, it doesn't preserve its original name, but rather prefixes it with `airlift_`. This means that the IR that the lifter emits does not have an entry point. Instead, the translator wrapper patches it with a new entry point that performs initialization, and only then calls the old entry point.

Inside the embedded entry point, control is transferred to the BTR, which allocates memory for a guest binary stack and initializes the state's stack pointer with a pointer to the end (since the stack grows from larger to smaller addresses) of this chunk of memory.

For reasons explained in detail in section 4.5, we include a copy of the binary we are translating inside our IR as an array, laid out as if it were loaded into memory by the loader.

4.3 Functions

To translate any meaningful program, we need to solve the problem of translating function calls. We will split all functions into two categories — regular functions and functions from shared libraries (external functions). Translating both categories involves solving different problems. After we have designed how our guest executable accesses the stack, we can solve the problem of function translation.

4.3.1 Function Declarations

Whenever our lifter encounters a symbol known to be a function (from its symbol table entry), it emits an LLVM IR function corresponding to it. The lifter then checks the function size in the symbol table to determine which instructions should be part of the function. Since we are emitting emulation-style IR, we have the advantage of not having to pass around function arguments, since they are stored in the state at the time of the call. The only required information to translate a function is a pointer to the state. Hence, all of the LLVM IR functions that the lifter will emit will have the signature `void fn(ptr)`, where ptr is the opaque pointer to the state.

The fact that we are not required to pass arguments around is a major advantage of emulation-style IR for simplicity of implementation. The alternative approach is to recover high-level function signatures heuristically from the function's binary data. This is not possible in the general case, for instance, because a function may legally ignore one of its arguments, and we would have no information at the binary level to recover it from. Other factors that contribute to the difficulty of this problem are type absence, variadic functions, and compiler optimizations.

4.3.2 Internal Function Calls

The second component of translating functions is translating calls to them. We first need to understand how calls and returns work on AArch64 and x86-64.

A subroutine call on AArch64 is triggered by the `bl LABEL` (branch-link) instruction. When executed, the `bl` instruction saves the address of the instruction after `bl` into `x30` and changes program counter (PC) to the address of the symbol passed to it. The `x30` register is responsible for holding the value to which the program should return after the function call. `ret` — the subroutine return instruction of AArch64 — takes advantage of this mechanism. When a `ret` instruction is executed, it changes the PC to the value of `x30`.

To perform the same actions on x86-64, one would have to invoke the `call LABEL` and `ret` instructions, respectively. The difference between the two architectures is in how they store the return value. While AArch64 has a dedicated register, x86-64 uses the stack for this. When a `call` instruction is invoked on x86-64, the PC is set to the address of the LABEL we are calling, and the address of the next instruction is pushed onto the stack. When a `ret` instruction is executed, a value is popped from the stack and the PC is set to it.

We illustrate the difference between calling mechanisms of the two architectures in figure 4.3. The figure shows the states of the stack and the link register (only for AArch64) side by side. On the left, the state immediately after a subroutine call is

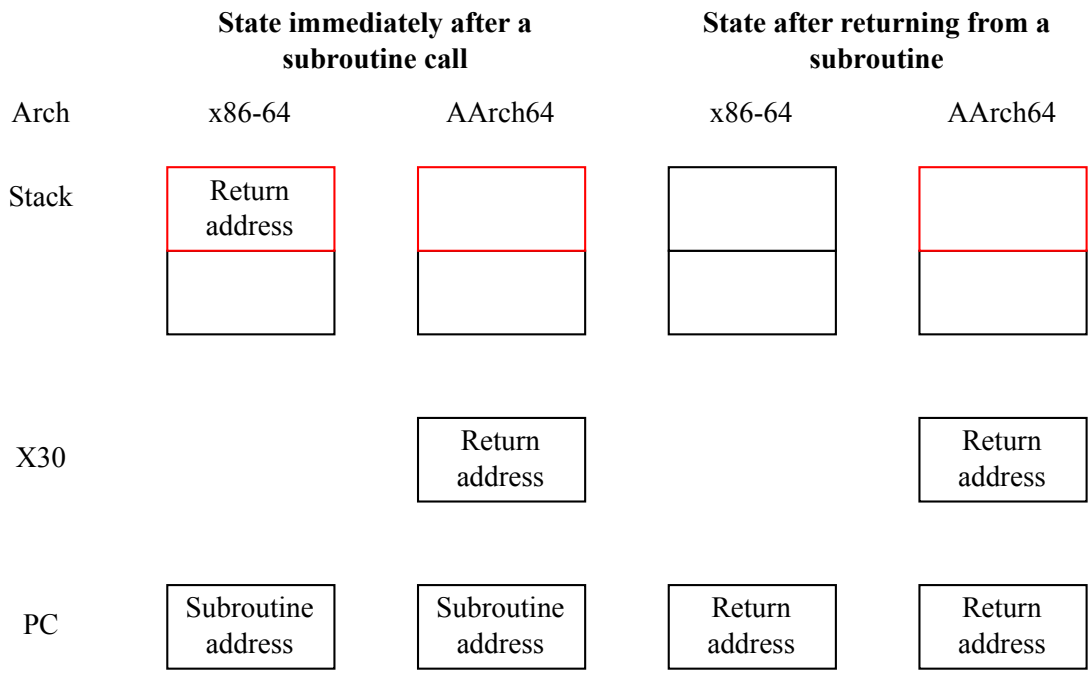

Figure 4.3: States of stack and `x30` (not present on x86-64) immediately after calling a subroutine and immediately after returning from a subroutine. The return address is the address of the next instruction after the call instruction. Subroutine address is the address of the symbol referred to by the call instruction. The cells with red outlines denote the cells the stack pointer points to. On AArch64, it is the first free cell. On x86-64, it is the last occupied cell.

shown. On the right, the state immediately after returning from a subroutine is shown.

Now that we understand the semantics of calls and returns in AArch64 and x86-64, we can proceed to translating them. Figure 4.4a shows two functions: `main` and `some_function`. Let `main` be the entry point of the code snippet. From `main`, the function `some_function` is called, which immediately returns.

Figure 4.4b shows a translation scheme for the call and the return. For the `main` function, we start by storing the address of the current PC in `guest_elf` into `x30`. This will not play a major role in how we handle calls, but is included for consistency, in case some code in the function refers to the return address in a way other than actually returning to it. After that, we emit an LLVM call instruction to `some_function`. We emit a call and not a branch, since LLVM IR does not support branching across functions. This is an important point, as the LLVM backend will lower this instruction into an

```
some_function:
ret

main:
bl some_function
...
```

(a) AArch64 snippet with functions `main` and `some_function`

```
define void some_function(ptr %state) {
  %x30_ptr = getelementptr { { i1, i1, i1, i1 }, [31 x i64], i64, i64, i64, i64, [32 x
    ↪ i2048], [16 x i256], i256 }, ptr %state, i32 0, i32 1, i32 30
  %x30_val = load i64, ptr %x30_ptr, align 8
  ret
}

define void main(ptr %state) {
  %x30_ptr = getelementptr { { i1, i1, i1, i1 }, [31 x i64], i64, i64, i64, i64, [32 x
    ↪ i2048], [16 x i256], i256 }, ptr %state, i32 0, i32 1, i32 30 ; Get ptr to x30
  store i64 add (i64 ptrtoint (ptr getelementptr inbounds ([131072 x i8], ptr @guest_elf,
    ↪  i64 0, i64 624) to i64), i64 4), ptr %state, align 8 ; Store current PC to x30
  call void @some_function(ptr %state)
}
```

(b) `main` and `some_function` lifted to LLVM IR

```
some_function:
ret

main:
... ; emulate the store of old PC into x30
call some_function
```

(c) `main` and `some_function` lowered to x86-64

Figure 4.4: The three snippets show what a function call looks like natively on AArch64, after lifting and after lowering to x86-64

x86-64 call as shown in figure 4.4c, which means that all of our function calls will use the x86-64 calling mechanism on the host stack.

As for the `ret` instruction from `some_function`, we translate it as a load from `x30` and an LLVM `ret`. In this case, the load from x30 is dead code and will be removed by the optimizer, since we yet again cannot branch back across functions. The LLVM `ret` will be translated into an x86-64 `ret`, which reads the return value from the stack,

```
string get_external_call_name(Instruction instr, CodeContext ctx) {
    if (!instr.is_call()) {
        return NULL;
    }

    Address target_address = instr.get_target_address();

    // Find index in PLT
    Index index_in_plt = get_index_in_plt(target_address);

    // If the target address is not in PLT, it's not an external call
    if (index_in_plt == -1) {
      return NULL;
    }

    // Through PLT relocation table, find symbol's index in dynamic symbol table
    Index index_in_dst = ctx.get_rela_plt().get_nth(index_in_plt).
    ↪ get_index_in_dynamic_symbol_table();
    return ctx.get_dynamic_symbol_table().get_name_of_entry(index_in_dst);
}

```

Figure 4.5: A pseudocode representation of an algorithm that decides if a call is external or not, and returns its name if it is

changes the PC to it, and pops it from the stack. This means after the subroutine, our host stack and guest stack are back to the expected, consistent state.

4.3.3 External Function Calls

Many binaries today are linked dynamically and not statically. SGTranslator supports translation of dynamically linked executables under certain conditions. If a user is willing to translate a dynamically linked executable, they have to provide the x86-64 version of the shared library and its headers to SGTranslator. The headers are used to restore function signatures when translating calls to external functions.

Handling external function calls requires cooperation between the lifter and the patcher (a part of the translator wrapper). Whenever a lifter encounters a call to an external function, it emits a placeholder call instead. Figure 4.5 shows pseudocode for determining whether a call is external.

To determine whether a function call is external, the lifter analyzes the source ELF's structure. Whenever the lifter encounters a `bl` instruction, it checks whether the target address is within the PLT. If so, the lifter recognizes the call as external. Airlift then

calculates the position of the symbol in PLT and looks for the entry with this index in the relocation table for PLT entries, `rela.plt`. From the relocation table entry, we can obtain the index of the symbol in the dynamic symbol table, which in turn yields the name of the external function being called. After finding the name of the function, lifter emits a placeholder similar to `call extcall_helper("name_of_the_function")`.

These placeholder entries will be handled by the IR patcher after the lifter has finished lifting. One of the responsibilities of the patcher is to resolve calls to `extcall_helper` into actual external function calls. This is why passing headers to the translator is necessary. The patcher parses the user-provided headers and analyzes all function signatures it finds. The most important data it extracts for every function is its name and its argument count.

After parsing headers, the patcher starts resolving all placeholders to real external function calls. Whenever SGTranslator encounters a placeholder, it removes it from the IR and emits a sequence similar to one shown in figure 4.6 in its place.

```
call void @airlift_extcall_helper(ptr @extcall_puts)
```

(a) An external call placeholder

```
%arg1_ptr = getelementptr %AirliftRegState, ptr %state, i32 0, i32 1, i32 0 ; retrieve
    ↪ pointer to x0 from state
%arg1_val = load i64, ptr %reg_ptr1, align 8 ; load the value from pointer to x0
%cast = inttoptr i64 %reg_val to ptr ; Cast it to const char*
%puts_ret = call i32 @puts(ptr %cast) ; Call puts with it as first argument
%ret_cast = zext i32 %puts_ret to i64 ; Cast return value of puts back to i64
%retreg_ptr = getelementptr %AirliftRegState, ptr %state, i32 0, i32 1, i32 0 ; retrieve
    ↪ pointer to x0
store i64 %ret_cast, ptr %retreg_ptr, align 8 ; store the returned value to x0
```

(b) Lifted code

Figure 4.6: Real external call sequence

In figure 4.6a, the placeholder has a pointer to a string containing the name of the external function that needs to be called. The patcher examines all signatures it encounters in headers and chooses the function whose name matches the string pointed to by the argument. It then adds an external declaration of this function to the module with its real arguments. After that, the patcher emits a sequence that extracts as many arguments as the function signature specifies from the state, casts them to the required types, and passes them to an actual function call of the function we just inserted into the module. After the call, the patcher casts the return value of this function to the

correct type and stores it back in the state.

The sequence can be seen in figure 4.6b. In the case shown in the figure, the function called is `puts`, a function from the C standard library. Its signature is `int puts(const char *str)`. `puts` takes a pointer to a null-terminated string as its first and only argument and prints it to the standard output. According to the calling convention of AArch64, the first argument passed to a function is from register `x0`. We retrieve a pointer to `x0` from the guest state and dereference it. Since every register in the state is stored as an `i64`, we have to cast this value to the correct type that `puts` expects — an opaque pointer (modern LLVM represents any pointer type, in our case, `const char*` as an opaque pointer type). After that, we can call `puts` with the opaque pointer we created as the first argument. The last thing we have to consider is that `puts` returns an integer — a non-negative value on success. This means we have to cast the i32 we got as the return value back to an i64, which we can then store in the state.

4.4 System Calls

Most binaries contain at least the exit system call to stop program execution, which means handling system calls is necessary to translate even the simplest programs. System calls are usually abstracted away at the C level, but they can still be invoked directly via inline assembly, represented in LLVM IR by the `call asm` construct. Lifting back to inline assembly is unsuitable for our purposes, as it produces AArch64-specific LLVM IR which cannot be lowered to x86-64. To circumvent this issue, the lifter lifts any system call it encounters to a helper function call (`airlift_syscall`), which will be made accessible when linking against the BTR.

At the guest binary runtime, where a syscall would have been executed in the original binary, `airlift_syscall` is called instead. To emulate the syscall on x86-64, `airlift_syscall` matches the value in `w8` (the AArch64 syscall number register) against the correct corresponding system call number on x86-64, invokes it with the required argument count, and updates `x0` with the return value. The reason `airlift_syscall` cannot forward the syscall number it receives and must map it is that the syscall numbers and the arguments syscalls take can differ across architectures. After invoking the correct syscall, the runtime returns control to the guest executable, and execution continues normally. By doing so, the BTR emulates the guest binary's system calls.

The reason that we have a separate BTR and do not patch `airlift_syscall` similarly to how we resolve external function calls is that we cannot determine what system call is being invoked from the `svc` (AArch64's syscall instruction) instruction alone. When we see an external function call, we can resolve the number of arguments it takes from its header. We do not have this information when decoding a syscall instruction. The

system call number is passed in the `w8` register, and based on its value, we decide the number of arguments that the syscall needs and which syscall to invoke.

4.5 Position-Independent Code

The next important class of instructions we need to handle specially in the lifter is position-independent code (PIC) instructions. Position-independent code is machine code that computes memory addresses relative to the PC, rather than using absolute addresses. This property is necessary for shared libraries, which may be loaded at different virtual addresses each time they are loaded. Many executables are also compiled as position-independent executables, which allows the operating system to utilize a protection mechanism called address space layout randomization (ASLR) [PaX03]. In many cases, position-independent executable (PIE) emission is enabled by default in compilers. For instance, the GCC binary distributed by Ubuntu has PIE enabled by default [Ubu24].

Figure 4.7a shows a C program that declares a static global variable, sets its value to 42, and returns its value as the exit code. On many Linux machines, the program will be compiled as a PIE by default.

Figure 4.7b shows the sequence the C program is compiled into. It starts with two stack management instructions, followed by an `adrp` instruction. `adrp` is the instruction responsible for making this code snippet PIC. On AArch64, `adrp` is an instruction that loads the address of a symbol's page relative to the current PC into a destination register. In our case, `adrp` loads the address of the page which contains `someglobal` into `x0`, relative to the current PC. After this, the only thing left to do to access `someglobal` is to add its offset (0x10) inside its page to `x0`. The program proceeds by loading from the computed address into `w0`, cleaning the stack, and returning. By emitting such code, the main symbol can be loaded anywhere into the memory by the loader, and the code will still work as expected. Having established how PIC arises, we now describe how the lifter handles it.

In order to handle PIC translation, we modify the lifter. We discussed in section 4.2 that the resulting binary contains a mapped copy of the original binary. The mapping is performed by the lifter before per-instruction lifting begins.

The lifter analyzes the source binary and embeds its loadable segments (we discussed segment loading in 2.3) into the LLVM IR module it outputs. To do it, we find the last `PT_LOAD` segment `lastsegment` and calculate `lastsegment.p_vaddr + lastsegment.p_memsize`, which is the amount of memory our program would take if it were loaded into RAM by the loader. We embed an array of this size into the IR. After allocating the array, we iterate over all loadable program headers and copy

```c
static int someglobal = 42;

int main() {
  return someglobal;
}
```

(a) A C program that returns the value of a static global variable as its exit status

```
0000000000020000 <data_start>:
  ...
0000000000020008 <__dso_handle>:
  20008: 08 00 02 00     .word 0x00020008
  2000c: 00 00 00 00     .word 0x00000000
0000000000020010 <someglobal>:
  20010: 2a 00 00 00     .word 0x0000002a

0000000000000824 <main>:
  824: a9bf7bfd       stp x29, x30, [sp, #-0x10]! ; Stack management
  828: 910003fd       mov x29, sp ; Stack management
  82c: 90000100       adrp x0, 0x20000 <data_start> ; Calculate address of the page
    ↪ containing someglobal relative to PC
  830: 91004000       add x0, x0, #0x10 ; Add offset of someglobal from start of the
    ↪ page
  834: b9400000       ldr w0, [x0] ; Load from &someglobal
  838: a8c17bfd       ldp x29, x30, [sp], #0x10 ; Stack management
  83c: d65f03c0       ret
```

(b) Output of `llvm-objdump` of the compiled C program

Figure 4.7: A code snippet which translates to a sequence with PIC

their contents into `guest_elf`, mimicking the loader. This process only works for ELF binaries, which means our translator is restricted to them.

The `guest_elf` array allows us to translate PIC with the following approach: Whenever we encounter a PIC instruction, we lift it such that it accesses data inside `guest_elf` and not inside the resulting binary. Figure 4.8 shows an example of a sequence `adr` would be translated into. `adr` is an instruction similar to `adrp`, with the difference that `adr DEST, SYMBOL` loads the complete address of a symbol relative to the PC into a destination register and not just the address of the page. The compiler cannot emit this instruction instead of `adrp` and `add` in all cases, since `adr` is limited to symbols not farther than 1 MiB away from the current PC due to the way it is encoded.

The instruction shown in 4.8a is expected to store the address of the label `SOME_LABEL` relative to the current PC into `x0` when executed natively. Instead of operating relative

```
adr x0, SOME_LABEL
```

(a) AArch64 adr instruction

```
%PC_PTR = getelementptr [SIZEOF_GUESTELF x i8], ptr @guest_elf, i64 0, i64 GUESTELF_PC
%PC_PTR_AS_INT = ptrtoint ptr %PC_PTR to i64
%EL_PTR = add i64 %PC_PTR_AS_INT, GUESTELF_OFFSET
store i64 %EL_PTR, ptr %X0_PTR, align 8
```

(b) Lifted code

Figure 4.8: Lifted AArch64 adr instruction

to the current PC, we operate relative to this instruction's offset in the `guest_elf` array.

When Airlift sees the `adr` instruction, it is translated to a sequence similar to one shown in figure 4.8b. `GUESTELF_PC` is the index of the instruction we are currently translating inside `guest_elf`. The lifter keeps track of this index during lifting. We index the `guest_elf` with it and obtain the address of `GUESTELF_PC`-th element. After that, we add `GUESTELF_OFFSET`, which is obtained from instruction encoding (and can also be negative), to it to get the correct address of the target symbol in `guest_elf` and store it into `state.x0`.

When translating PIC with this approach, loading the program segmentwise is required, since the instructions refer to virtual addresses that assume the program has been loaded into memory as specified in the program header table.

The upside of this approach is its simplicity: one does not have to worry about lifting `.data` or `.bss` sections separately. They are a part of the loadable segment and are therefore copied into our LLVM IR module via `guest_elf`. The downside is that our resulting program is now at least as large as the source program (and in practice, twice as large), which is especially problematic for large executables. Due to page alignment and zero-initialized data, such as `.bss` section, which does not take up space before the executable is mapped into the memory by the loader, the `guest_elf` array can have lots of zeroed out data, which is never accessed.

4.6 Specifics of C translation

Our translator is not completely language-agnostic. Before lifting, the lifter iterates over all function symbols it can find in the guest binary and checks if any of them are C maintenance functions (e.g., `__libc_start_main` — libc's initialization function). If a C maintenance function is found, the lifter treats `main` as the binary's entry point and

does not translate any functions that are related to C runtime initialization. The reason for this design decision is that C maintenance functions contain dynamic jumps and other features that the lifter does not support. Instead of translating them, they are re-linked by the **translator wrapper** along with the C runtime. If the lifter does not discover any functions that are marked as C maintenance functions, it will lift every function and treat `_start` as the entry point of the program. This ensures that our binary translator can translate pure assembly programs.

4.7 Limitations

Due to the nature of static binary translation, we encounter certain natural barriers that we cannot overcome. We also leave out some features that would be otherwise plausible to implement due to time constraints.

The features that we leave out include:

- Lifting of dynamic jumps (jumps to registers): static binary translators have difficulty with this problem in general due to a lack of context. This limitation prevents us from translating many high-level C concepts, such as switch constructs and function pointers.
- Lifting of vector and floating point registers: Airlift does not support lifting these classes of instructions.
- Translation of programs written in languages other than C and assembly.
- Translation of external calls to variadic functions.
- Translation of external functions which pass structures or unions as arguments by value or by pointer: The General case is out of scope of this thesis, since structures and unions can contain floating point types, which are not supported by the lifter.
- Translation of global variables with external linkage.

5 Implementation

```
Usage: sgtranslator [--help] [--version] [--output VAR] [--dump-after-all] [--print-
    ↪ after-all] [--config VAR] [--no-opt] [--preserve-obj] input

Positional arguments:
  input                   Name of the input file

Optional arguments:
  -h, --help              shows help message and exits
  -v, --version           prints version information and exits
  -o, --output            Output file name
  -d, --dump-after-all    Dump after all stages
  -p, --print-after-all   Print after all stages
  -c, --config            Path to config file (default: ./config.json)
  --no-opt                Skip the optimization stage
  --preserve-obj          Preserve the intermediate .o after linking
```

Figure 5.1: The help message of SGTranslator

As our project strongly relies on LLVM, it is a natural choice to implement both the translator wrapper and the BTR in C++, the language of the LLVM Project. We use LLVM to patch, optimize, and lower the IR we obtain from Airlift and use LLVM's lld [The26a] as a library as our project's linker. The optimizations we perform on the IR we obtain from the lifter correspond to a typical `-O2` pipeline. SGTranslator only supports translation of ELF executables, since the modules responsible for resolving external function calls and laying out the `guest_elf` consider the binary to be an ELF binary.

Our second large dependency is Airlift itself. Airlift is written in Rust, with sequences responsible for generating LLVM IR also being in Rust, which is generated by parsing ARM's MRAS. In order to expose Rust to SGTranslator, we write a C interface for it, consisting of two functions — `airlift_lift` and `airlift_free_string` and compile it as a dynamic C library. `airlift_lift` takes the path of the guest binary as an input and returns a string containing the lifted binary's IR. `airlift_free_string` is used to free that string to avoid a memory leak.

The header-parsing module, required for external function call translation, is a

modified version of mctoll's header-parsing module [Mic26]. We add an implicit error-invocation mechanism that triggers when the user attempts to translate a variadic function or a function that takes a struct by value. We modify mctoll's header parser to resolve header include dependencies by scanning their parent folders.

SGTranslator requires many input arguments. To avoid passing them from the command line, we implement a JSON configuration system that allows the user to specify the paths to dynamic library dependencies and their headers, the paths to C runtime dependencies, and the path to BTR.

Figure 5.1 shows the help message of SGTranslator. We allow printing the results to stdout after all stages of translation (before patching, after patching, after optimization) or dumping them into separate files for better debuggability. We also allow skipping LLVM IR optimization and preserving an unlinked object file emitted by codegen.

6 Evaluation

In this chapter, we evaluate the correctness, the speed, and the executable size of binaries produced by SGTranslator when translating from AArch64 to x86-64.

6.1 Performance

We first investigate the performance of the binary translator using data collected from PolyBenchC. PolyBenchC is a collection of 30 numerical kernels with static control flow, drawn from domains such as linear algebra, image processing, physics simulation, dynamic programming, and statistics [PY16].

6.1.1 Methodology

We modify PolyBenchC to suit our needs by rewriting parts that call variadic functions; instead of using `fprintf` for dumping results to stdout, we use `write`. During benchmarking, we disable prints. We still need to adapt them to using `write`, since we use the same benchmarks to evaluate the correctness of SGTranslator, which requires printing for performing output comparisons. We also modify the initialization procedures of our benchmarks to avoid floating-point computations, since SGTranslator does not support translating them. For the same reason, we include the initialization time of benchmarks in the performance results — PolyBenchC internally relies on a floating-point timer.

We filter out the benchmarks that our translator does not support. Our final benchmark suite includes 15 benchmarks: the basic linear algebra subprograms (BLAS), as well as doitgen, nussinov, and floyd-warshall benchmarks. Figure 6.1 shows a table of all benchmarks PolyBenchC offers, along with those we cannot evaluate because they do not produce meaningful results without enabling floating-point computations.

All experiments are conducted on a server-grade x86-64 machine equipped with an AMD EPYC 7713P processor with 64 cores, each running up to 3.72 GHz. The processor provides 2 MiB of L1 instruction/data cache per core, 32 MiB of L2 cache, and 256 MiB of L3 cache within a single NUMA node. Memory pressure is negligible due to the system's 1 TiB of RAM. To ensure consistent measurements, we execute all experiments on a single core, avoiding parallelism and inter-thread cache interference.

Category	Benchmark	Included
Lin. Alg. Kernels	2mm	✓
	3mm	✓
	atax	✓
	bicg	✓
	doitgen	✓
	mvt	✓
Lin. Alg. BLAS	gemm	✓
	gemver	✓
	gesummv	✓
	symm	✓
	syr2k	✓
	syrk	✓
	trmm	✓
Lin. Alg. Solvers	cholesky	×
	durbin	×
	gramschmidt	×
	lu	×
	ludcmp	×
	trisolv	×
Datamining	correlation	×
	covariance	×
Medley	deriche	×
	floyd-warshall	✓
	nussinov	✓
Stencils	adi	×
	fdtd-2d	×
	heat-3d	×
	jacobi-1d	×
	jacobi-2d	×
	seidel-2d	×

Table 6.1: PolyBenchC 4.2.1 benchmark selection. 15 of 30 benchmarks were included. Excluded benchmarks rely on floating-point arithmetic, which the translator does not support.

We start by compiling the benchmarks for native x86-64. We compile the benchmarks with `gcc -O2 -fno-stack-protector -mgeneral-regs-only -DLARGE_DATASET`, restricting the compiler to only using general registers, since the lifter does not support other classes of registers. After compiling the benchmarks for x86-64, we compile them for AArch64 using a cross-compiler. The compilation options are identical to the ones used for native compilation. We proceed by translating the AArch64 binaries to x86-64 with

SGTranslator. During translation, we run optimization passes on the IR generated by the lifter, which correspond to a typical `-O2` optimization pipeline. We generate three types of data for 30 runs: execution time of the native binary, execution time of the binary produced by SGTranslator, and execution time of the AArch64 with QEMU (`qemu-aarch64`).

We evaluate the overhead experienced by a user running an AArch64 binary via SGTranslator on x86-64, compared to executing a natively compiled x86-64 binary and using QEMU.

6.1.2 Results

To facilitate visualization, we divide the results into three datasets with comparable runtimes and generate a chart for each category.

The results are presented in figures 6.1, 6.2, and 6.3. The x-axis shows the different benchmarks, and the y-axis shows execution time. Blue bars indicate the execution time of a benchmark compiled natively for x86-64, while orange bars indicate the execution time of the translated benchmark, and green bars indicate execution time with QEMU. The black segments represent the standard deviation: for each bar, a vertical black line is centered on the mean of 30 runs and extends one standard deviation above and below it. The numbers above the bars for SGTranslator and QEMU indicate the overhead relative to native execution. For the slow benchmarks, the Floyd-Warshall benchmark is a runtime outlier. To preserve the scale of other results, we truncate the chart at 20 seconds and resume it at 90 seconds.

The geometric mean of the runtime ratios

$$\frac{t_{\text{translated}}}{t_{\text{native}}}$$

is 3.686. This means on average, the runtime of a translated binary is 3.686 times that of a native binary. Our best result comes from the `gesummv` benchmark, with a slowdown of 1.680, and our worst result is the Floyd-Warshall benchmark with a slowdown of 7.577.

For the smaller tests, SGTranslator outperforms QEMU, whereas for larger tests, SGTranslator is slower. This can be attributed to QEMU initialization-time overhead, which dominates runtime for smaller benchmarks but becomes negligible for larger ones.

We also measured the sizes of the binaries produced by SGTranslator for each benchmark. This is depicted in table 6.2.

The table shows the sizes of the original AArch64 binaries, the sizes of translated x86-64 binaries, and the size ratio. Since all PolyBenchC benchmarks link with the large

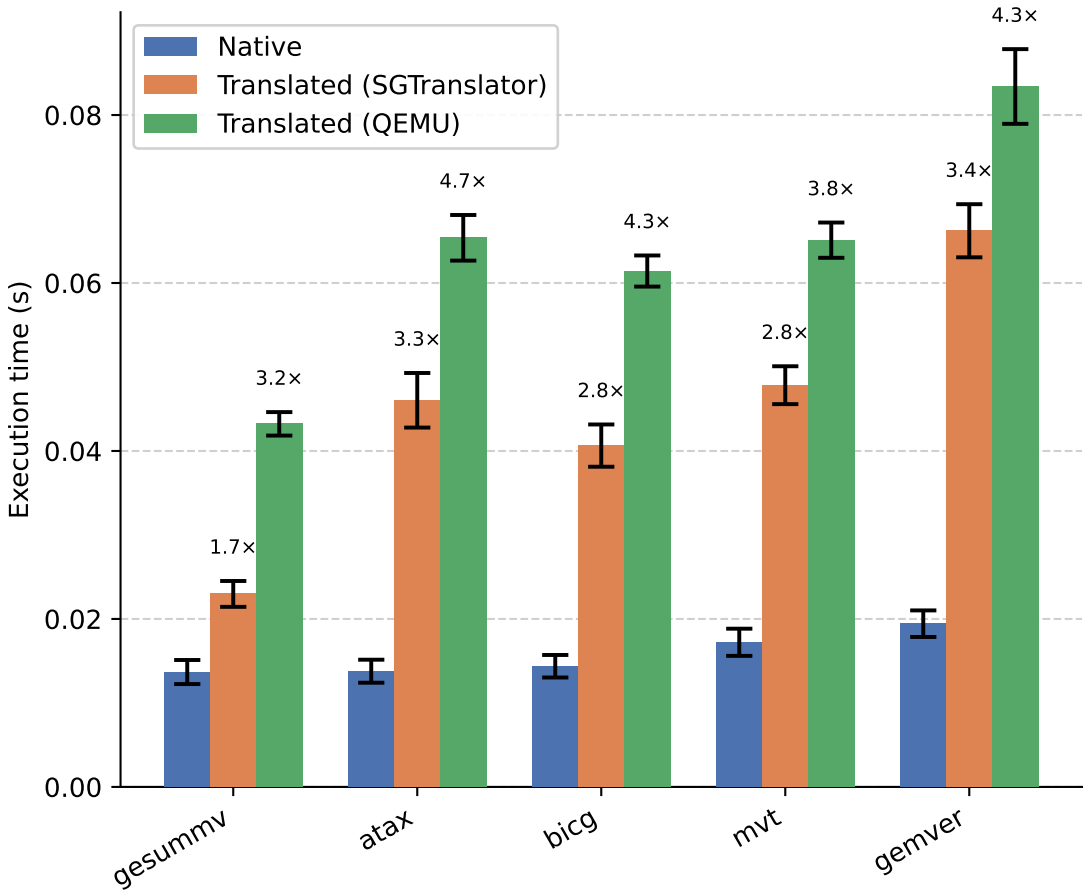

Figure 6.1: Performance results for tiny benchmarks

helper file `polybench.c`, the overall sizes of the benchmarks are roughly uniform. As expected, all binaries are at least twice the size of the original binary since we embed `guest_elf` into the result binary.

The translated binaries are consistently approximately 2.22× the size of the native binaries. The low variance in size ratios (2.19×–2.31×) suggests that the overhead is largely constant and dominated by the embedding of `guest_elf` into the resulting binary, rather than by the size of the translated code itself.

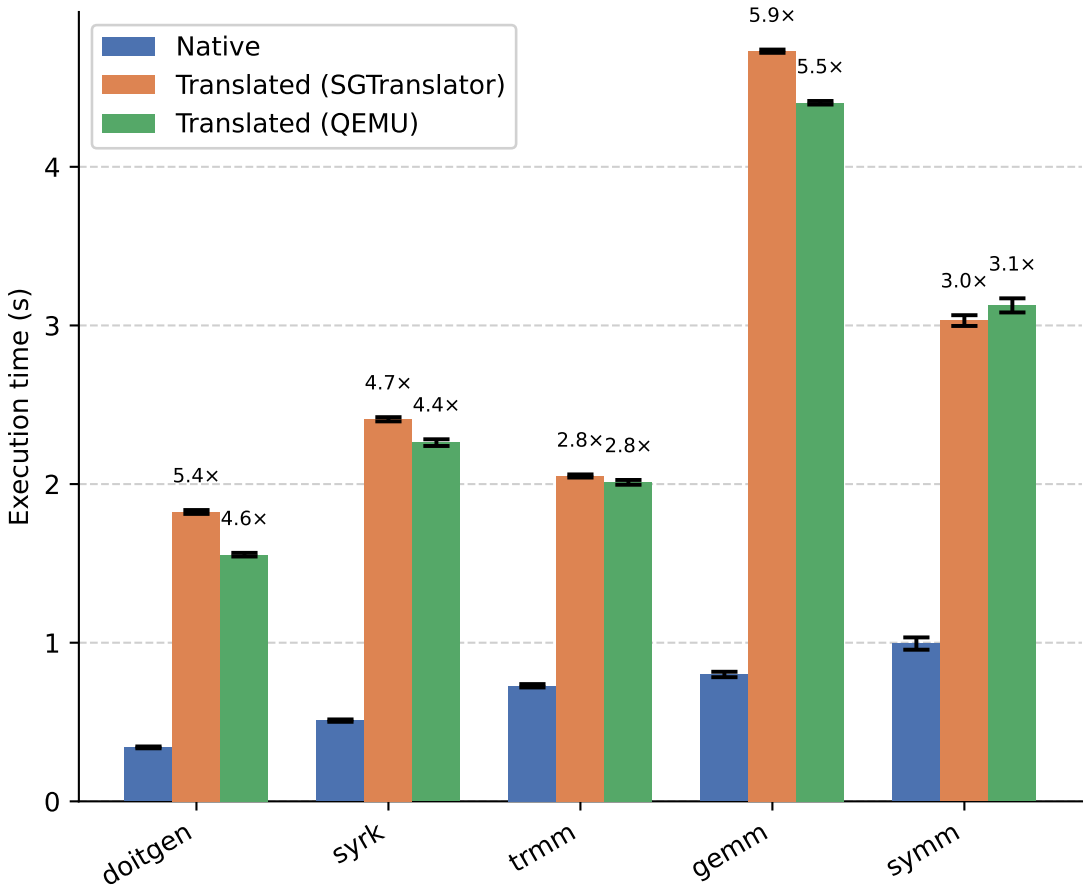

Figure 6.2: Performance results for fast benchmarks

6.2 Correctness

To evaluate the correctness of SGTranslator, we develop a Python-based test runner. The test runner runs AArch64 binaries with QEMU, runs the translated version, and compares the exit codes, stdouts, and stderrs. QEMU is an open-source DBT that allows running binaries compiled for foreign architectures natively [Bel05].

To test the correctness of SGTranslator, we write a test suite. It contains per-instruction assembly tests to ensure basic correctness, as well as conceptual assembly tests for function calls, PIC handling, and assembly-level external function call handling. We then write several C tests to verify that the translator correctly executes programs compiled from C code. The largest of them is a mergesort test. We also run the

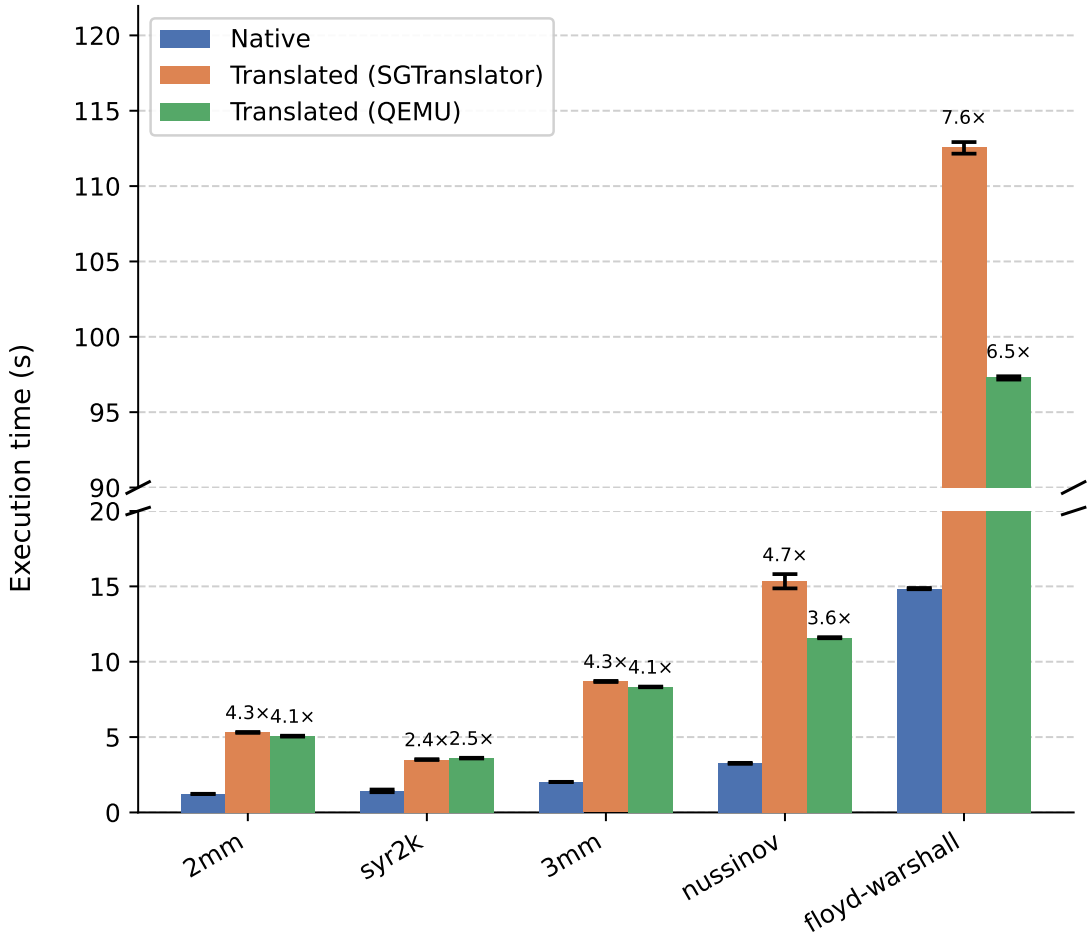

Figure 6.3: Performance results for slow benchmarks

entire PolyBenchC benchmark with enabled prints through the test runner and find no mistranslations when compared with QEMU.

We have a few correctness issues that we could not resolve due to time constraints. The first issue is the translation of tail-recursive functions that are optimized with the tail call optimization. The code emitted by the optimizer unconditionally branches to symbols that are function names, which causes the lifter to emit an inter-function LLVM `br`, which is ill-formed LLVM IR.

The second correctness issue arises when translating global variables with external linkage. The problem with their translation is that they can be routed through the GOT, which is filled in by the dynamic linker at load time when executing natively. In

Group	Benchmark	AArch64 (KiB)	x86 (KiB)	Size ratio
BLAS-2 (matrix-vector)	`atax`	69.3	155.6	2.24×
	`bicg`	69.3	155.6	2.25×
	`gemver`	69.3	159.7	2.31×
	`gesummv`	69.3	155.6	2.25×
	`mvt`	69.3	155.6	2.25×
BLAS-3 (matrix-matrix)	`2mm`	69.3	155.6	2.25×
	`3mm`	69.3	155.6	2.25×
	`gemm`	69.3	151.5	2.19×
	`symm`	69.3	151.5	2.19×
	`syr2k`	69.3	151.5	2.19×
	`syrk`	69.3	151.5	2.19×
	`trmm`	69.3	151.5	2.19×
Other	`doitgen`	69.3	155.5	2.24×
	`floyd-warshall`	69.3	151.4	2.19×
	`nussinov`	69.3	151.5	2.19×
Geometric mean size ratio: 2.22×				

Table 6.2: AArch64 vs. translated x86 binary sizes for PolyBenchC benchmarks compiled with `-O2`.

our case, since the GOT is located in `guest_elf`, this does not happen, and we get a runtime crash instead.

7 Related Work

To the best of our knowledge, SGTranslator is the first end-to-end binary translator from AArch64 to x86-64 that uses a specification-driven lifter based on MRAS.

Tools based on MRAS existed before SGTranslator. A notable example is the ASLi (an ASL interpreter), developed by A. Reid. ASLi allows evaluation of arbitrary ASL statement sequences and expressions. This includes evaluating the whole ARM specification, which is essentially ARM simulation using ASLi [Rei20]. However, ASLi is an interpreter and not a translator, and does not target any specific architecture.

Other notable lifters from AArch64 to LLVM IR exist, but they are not specification-driven. These include llvm-mctoll [Mic26] and McSema [lif22]. Both are manually written, ahead-of-time binary lifters that support raising AArch64 to LLVM IR. llvm-mctoll is part of Microsoft's open-source projects, whereas McSema was developed by Trail of Bits and funded by the US Department of Defense. McSema is currently unmaintained, as the authors have moved on to remill, a library for binary lifting [Tra15]. Both solve the problem of lifting binaries into LLVM IR, but manually developing such tools is error-prone and does not scale well.

Another project that uses LLVM IR to translate from x86-64 to AArch64 (but is not limited to this pair) is A. Engelke's instrew. It is a dynamic binary translator that lifts code to LLVM IR, modifies it, and lowers it to the target architecture using LLVM's JIT compiler [Eng21].

Another well-established dynamic binary translator is QEMU, which we use as a reference point in our evaluation section (section 6). QEMU supports translation between a variety of architectures and runs on multiple operating systems. Both full-system emulation and dynamic binary translation (user-mode emulation) are supported [Bel05].

Several projects have tackled translation in the opposite direction. One of such projects is Risotto, a dynamic binary translator built on top of QEMU. It solves a problem inverse to ours: the translation from x86-64 to ARM. The problem is technically challenging because ARM architectures have a weak memory model, whereas x86-64 has a strong one. Risotto's contribution lies in formalizing the memory model of the TCG IR (QEMU's internal representation) and proposing formally verified mapping schemes from a strong architecture to a weak architecture [Gou+22].

The Lasagne project is the static counterpart of Risotto. It is a static binary translator

that also translates from x86-64 to ARM, aiming to solve the problem of emulating concurrent binaries from a strong architecture on a weaker one [Roc+22].

Arancini is a hybrid binary translator that aims to solve the same problems as Lasagne and Risotto, combining the best of DBT and SBT [Rei+26].

All of these projects have contributed to the field of binary translation, yet we are the first to propose a complete static binary translator from AArch64 to x86-64 based on MRAS and LLVM IR.

8 Conclusion

During the course of this thesis, we developed SGTranslator, a static binary translator based on Airlift that translates AArch64 to x86-64. We developed a translator wrapper that orchestrates all translator components, along with a helper runtime library. These components are essential to build a complete binary translator on top of Airlift, which we modified to suit our needs. SGTranslator can successfully translate a large set of C programs and pure assembly programs. It successfully demonstrates that specification-driven binary translators are a plausible solution to the problem of static binary translation.

The most significant technical challenges solved during the course of the thesis are the translation of functions from shared libraries, the translation of position-independent code, the correct initialization of the guest binary's state, and the emulation of system calls.

8.1 Future Work

Possible future work could involve extending the feature set of SGTranslator with features left out due to time constraints (discussed in section 4.7), as well as addressing the correctness issues we discussed in section 6.2, such as the translation of global variables with external linkage and the optimization of tail-recursive functions.

In addition, the translator could be extended to support binary formats other than ELF, such as Mach-O and PE, and to detect binary hardening features, such as stack protection, and account for them during translation to produce correct binaries. Another significant extension that could be made is the translation of programs compiled from languages other than C.

Abbreviations

ELF executable and linkable format

PLT procedure linkage table

GOT global offset table

CISC complex instruction set computer

RISC reduced instruction set computer

ISA instruction set architecture

IR intermediate representation

BTR binary translator runtime

ASL Architecture Specification Language

MRAS Machine Readable Architecture Specification

PIC position-independent code

PIE position-independent executable

PC program counter

ASLR address space layout randomization

SBT static binary translator

DBT dynamic binary translator

List of Figures

2.1 Loading of a segment according to a program header. S1 and S2 are sections, **S** is the segment containing them. The filled red part of **V** is the part of the virtual segment that is zeroed out, since it is not backed by data from the file segment. 7

3.1 Overview of the translator workflow 10

4.1 A simplified version of a processor state 11
4.2 Lifted ARM add 12
4.3 States of stack and `x30` (not present on x86-64) immediately after calling a subroutine and immediately after returning from a subroutine. The return address is the address of the next instruction after the call instruction. Subroutine address is the address of the symbol referred to by the call instruction. The cells with red outlines denote the cells the stack pointer points to. On AArch64, it is the first free cell. On x86-64, it is the last occupied cell. 15
4.4 Comparison between ARM, LLVM and x86 16
4.5 A pseudocode representation of an algorithm that decides if a call is external or not, and returns its name if it is 17
4.6 Real external call sequence 18
4.7 A code snippet which translates to a sequence with PIC 21
4.8 Lifted ARM adr 22

5.1 The help message of SGTranslator 24

6.1 Performance results for tiny benchmarks 29
6.2 Performance results for fast benchmarks 30
6.3 Performance results for slow benchmarks 31

List of Tables

6.1 PolyBenchC 4.2.1 benchmark selection. 15 of 30 benchmarks were included. Excluded benchmarks rely on floating-point arithmetic, which the translator does not support. 27
6.2 AArch64 vs. translated x86 binary sizes for PolyBenchC benchmarks compiled with `-O2`. 32

Bibliography

[App20] Apple Corporation. *Apple unleashes M1*. Nov. 2020.

[Bel05] F. Bellard. "QEMU, a Fast and Portable Dynamic Translator." In: Jan. 2005, pp. 41–46.

[Cho25] W. Choi. "Airlift: A Binary Lifter Based on a Machine-Readable Architecture Specification." MA thesis. Technical University of Munich, 2025.

[CM96] Cifuentes and Malhotra. "Binary translation: static, dynamic, retargetable?" In: *1996 Proceedings of International Conference on Software Maintenance*. 1996, pp. 340–349. DOI: 10.1109/ICSM.1996.565037.

[Eng21] A. F. Engelke. "Optimizing Performance Using Dynamic Code Generation." Submitted 23.06.2021, accepted 28.10.2021. Dissertation. Technische Universität München, Oct. 2021.

[GNU] GNU Foundation. *GIMPLE*.

[Gou+22] R. Gouicem, D. Sprokholt, J. Ruehl, R. C. O. Rocha, T. Spink, S. Chakraborty, and P. Bhatotia. "Risotto: A Dynamic Binary Translator for Weak Memory Model Architectures." In: *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ASPLOS 2023. Vancouver, BC, Canada: Association for Computing Machinery, 2022, pp. 107–122. ISBN: 9781450399159. DOI: 10.1145/3567955.3567962.

[H J+21] H. J. Lu, M. Matz, M. Girkar, J. Hubička, A. Jaeger, and M. Mitchell. *System V Application Binary Interface: AMD64 Architecture Processor Supplement (With LP64 and ILP32 Programming Models)*. Version 1.0. The Open Group / System V ABI Committee. Sept. 2021.

[IBM16] IBM. *When to use dynamic linking and static linking*. IBM Knowledge Center. IBM Corporation. 2016.

[Int24] Intel Corporation. *Intel® Intrinsics Guide*. Version 3.6.9, updated July 12, 2024. Intel Corporation. 2024.

[LA04] C. Lattner and V. Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation." In: *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO'04)*. Palo Alto, California, Mar. 2004.

[lif22] lifting-bits. *McSema: Framework for lifting x86, amd64, AArch64, SPARC32, and SPARC64 binaries to LLVM bitcode*. `https://github.com/lifting-bits/mcsema`. Archived repository. 2022.

[Liu+22] Z. Liu, Y. Yuan, S. Wang, and Y. Bao. "SoK: Demystifying Binary Lifters Through the Lens of Downstream Applications." In: *2022 IEEE Symposium on Security and Privacy (SP)*. 2022, pp. 1100–1119. DOI: `10.1109/SP46214.2022.9833799`.

[Mic26] Microsoft. *llvm-mctoll: A Tool for Raising Binaries to LLVM IR*. `https://github.com/microsoft/llvm-mctoll`. GitHub repository. 2026.

[Ora10] Oracle Corporation. *Linker and Libraries Guide, Chapter 7: Object File Format*. 2010. URL: `https://docs.oracle.com/cd/E19683-01/816-1386/6m7qcoblh/index.html` (visited on 02/23/2026).

[PaX03] PaX Team. *PaX Address Space Layout Randomization (ASLR)*. `https://pax.grsecurity.net/docs/aslr.txt`. 2003.

[Pro02] M. Probst. *Dynamic Binary Translation*. Technical Report. Available at `https://www.complang.tuwien.ac.at/cd/papers/bintrans.pdf`. CD Laboratory for Compilation Techniques, Technische Universität Wien, 2002.

[PY16] L.-N. Pouchet and T. Yuki. *PolyBench/C benchmark suite, version 4.2.1*. Online; GitHub repository. Available at: `https://github.com/MatthiasJReisinger/PolyBenchC-4.2.1`. 2016.

[Rei+26] S. Reimers, D. Sprokholt, M. Fink, T. Augoustis, S. Kammermeier, R. C. O. Rocha, T. Spink, S. Chakraborty, and P. Bhatotia. "Arancini: A Hybrid Binary Translator for Weak Memory Model Architectures." In: ASPLOS 2026 (to appear). 2026.

[Rei17] A. Reid. *ARM releases machine readable architecture specification*. Apr. 2017.

[Rei20] A. Reid. *Using ASLi with Arm's v8.6-A ISA Specification*. `https://alastairreid.github.io/using-asli/`. Blog post. Jan. 2020.

[Roc+22] R. C. O. Rocha, D. Sprokholt, M. Fink, R. Gouicem, T. Spink, S. Chakraborty, and P. Bhatotia. "Lasagne: a static binary translator for weak memory model architectures." In: *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. PLDI 2022. San Diego, CA, USA: Association for Computing Machinery, 2022, pp. 888–902. ISBN: 9781450392655. DOI: 10.1145/3519939.3523719.

[She+12] B. Y. Shen, J. Y. You, W. Yang, and W. C. Hsu. "An LLVM-based hybrid binary translation system." In: *Proceedings of the 7th IEEE International Symposium on Industrial Embedded Systems (SIES 2012)*. Karlsruhe, Germany: IEEE, 2012, pp. 229–236. DOI: 10.1109/SIES.2012.6356589.

[The26a] The LLVM Project. *LLD – The LLVM Linker*. https://lld.llvm.org/. LLVM Project, 2026.

[The26b] The LLVM Project. *LLVM Language Reference Manual*. LLVM Foundation. 2026.

[Tra15] Trail of Bits. *Remill: Library for Lifting Machine Code to LLVM Bitcode*. https://github.com/lifting-bits/remill. Accessed: 2026-03-04. 2015.

[Ubu24] Ubuntu Wiki. *ToolChain/CompilerFlags*. Canonical Ltd. 2024. URL: https://wiki.ubuntu.com/ToolChain/CompilerFlags.